

Unit-4 Immutable Data Structures

Definition of Immutable Data Structure:

Immutable Data types are those that can never change their value in place.

Examples:

String, Tuple etc.

STRING

Definition of String:

- ❖ Any sequence of characters within either single quotes or double quotes is considered as a String.
- ❖ **Syntax:**
- ❖ `s = 'Arman'`
- ❖ `s = "Dhairya"`
- ❖ We can define multi-line String literals by using triple single or double quotes.
- ❖ **Example:**
- ❖ `s = '''Bharat`
`Gas'''`

Accessing Characters of String:

- ❖ We can access characters of a string by using the following ways.
- ❖ **(1) By using index**
- ❖ **(2) By using slice operator**

(1) Accessing Characters By using Index:-

- ❖ Python supports both +ve and -ve Index.
- ❖ +ve Index means Left to Right (Forward Direction)
- ❖ -ve Index means Right to Left (Backward Direction)
- ❖ **Example:**
- ❖ `s = "Arman"`
- ❖ `s[3] -> 'a'`
- ❖ `s[-1] -> 'n'`
- ❖ `s[9] -> IndexError: string index out of range`

(2) Accessing Characters by using Slice Operator: -

- ❖ **Syntax: s[beginindex:endindex:step]**
- ❖ **Begin Index:** From where we have to consider slice.
- ❖ **End Index:** We have to terminate the slice at endindex-1
- ❖ **Step:** Incremented Value.
- ❖ If we are not specifying begin index then it will consider from beginning of the string.
- ❖ If we are not specifying end index then it will consider up to end of the string.
- ❖ The default value for step is 1.
- ❖ Step value can be either +ve or -ve.
- ❖ If +ve then it should be forward direction(left to right) and we have to consider begin to end-1.
- ❖ If -ve then it should be backward direction (right to left) and we have to consider begin to end+1.
- ❖ **Example:**
- ❖ `s="Learning C is very very easy!!!"`
- ❖ `s[1:7:1] -> 'earnin'`
- ❖ `s[1:7] -> 'earnin'`
- ❖ `s[1:7:2] -> 'eri'`
- ❖ `s[:7] -> 'Learnin'`
- ❖ `s[9:] -> 'C is very very easy!!!'`
- ❖ `s[:] -> 'Learning C is very very easy!!!'`
- ❖ `s[:] -> 'Learning C is very very easy!!!'`
- ❖ `s[::-1] -> '!!!ysae yrev yrev si C gninrael'`
- ❖ `s[:] -all , s[:] -all, s[::-1] -Reverse`

Mathematical Operators for String:

- ❖ We can apply the following mathematical operators for Strings.
- ❖ **+ operator for concatenation.**
- ❖ *** operator for repetition.**
- ❖ **Example:**
- ❖ `print("Arman"+"Dhairya") -> ArmanDhairya`

- ❖ `print("Lucky"*2) -> LuckyLucky`
- ❖ To use + operator for Strings, compulsory both arguments should be str type.
- ❖ To use * operator for Strings, compulsory one argument should be str and other argument should be int.

Comparison of String:

- ❖ We can use comparison operators (`<=`, `>`, `>=`) and equality operators (`==`, `!=`) for strings.
- ❖ Comparison will be performed based on alphabetical order.
- ❖ **Example:**
- ❖ `s1=input("Enter first string:")`
- ❖ `s2=input("Enter Second string:")`
- ❖ `if s1==s2:`
- ❖ `print("Both strings are equal")`
- ❖ `elif s1<s2:`
- ❖ `print("Second String is greater")`
- ❖ `else:`
- ❖ `print("First String is greater")`
- ❖ **Output:**
- ❖ Enter first string: Arman
- ❖ Enter Second string: Arman
- ❖ Both strings are equal
- ❖ Enter first string: Arman
- ❖ Enter Second string: Lucky
- ❖ Second String is greater
- ❖ Enter first string: Dhairya
- ❖ Enter Second string: Arman
- ❖ First String is greater

Joining of String:

- ❖ We can join a Group of Strings (List OR Tuple) wrt the given Seperator.
- ❖ `s = seperator.join(group of strings)`
- ❖ **Example:**
- ❖ `t = ('Arman', 'Lucky', 'Dhairya')`
- ❖ `s = '-'.join(t)`

```
print(s)
```

❖ **Output:**

Arman-Lucky-Dhairya

Formatting of String:

- ❖ We can format the strings with variable values by using replacement operator {} and format() method.

❖ **Example:**

```
name = "Arman"
salary = 100000
age = 24
print("{}'s salary is {} and his age is {}".format(name,salary,age))
print("{0}'s salary is {1} and his age is {2}".format(name,salary,age))
```

❖ **Output:**

Arman's salary is 100000 and his age is 24
Arman's salary is 100000 and his age is 24

Important Functions of String:

❖ **len() Function:**

- ❖ **len() function** returns the length of the string.

- ❖ **Syntax:** len(string)

- ❖ **Return:** It returns an integer which is the length of the string.

❖ **Example:**

```
string = "LJIET"
print(len(string))
```

❖ **Output:** 5

❖ **Removing Spaces from String:**

- (1) **rstrip():** To remove spaces at right hand side

❖ **Example**

```
txt = "banana"
x = txt.rstrip('a')
print("of all fruits", x, "is my favorite")
```

❖ **Output:** of all fruits banan is my favorite

(2) lstrip():To remove spaces at left hand side

❖ **Example**

Remove the leading characters:

```
txt = "banana"  
x = txt.lstrip("b")  
print(x)
```

❖ **Output:** anana

(3) strip():To remove spaces both sides

❖ **Example**

```
txt = " banana "  
x = txt.strip()  
print("of all fruits", x, "is my favorite")
```

❖ **Output:** of all fruits banana is my favorite

❖ **Changing Case of String:**

❖ We can change case of a string by using the following five methods.

(1) upper(): To convert all characters to upper case

❖ **Example**

```
txt = "Hello my friends"  
x = txt.upper()  
print(x)
```

❖ **Output:** HELLO MY FRIENDS

(2) lower(): To convert all characters to lower case.

❖ **Example**

```
txt = "Hello my FRIENDS"  
x = txt.lower()  
print(x)
```

❖ **Output:** hello my friends

(3) swapcase(): Converts all lower case characters to upper case and all upper case characters to lower case.

❖ **Example**

```
txt = "Hello My Name Is PETER"  
x = txt.swapcase()  
print(x)
```

❖ **Output:** hELLO mY nAME iS peter

(4) title(): To convert all character to title case. i.e first character in every word should be upper case and all remaining characters should be in lower case.

❖ **Example**

```
txt = "Welcome to my world"  
x = txt.title()  
print(x)
```

❖ **Output:** Welcome To My World

(5) capitalize(): Only first character will be converted to upper case and all remaining characters can be converted to lower case.

❖ **Example**

```
txt = "hello, and welcome to my world."  
x = txt.capitalize()  
print(x)
```

❖ **Output:** Hello, and welcome to my world.

❖ **Example:**

```
s = 'learning C is very Easy'  
print(s.upper())  
print(s.lower())  
print(s.swapcase())  
print(s.title())
```

```
print(s.capitalize())
```

❖ **Output:**

LEARNING C IS VERY EASY

learning c is very easy

LEARNING c IS VERY eASY

Learning C Is Very Easy

Learning c is very easy

❖ **To Check Type of Characters Present in a String (Check Function): Answer only in True of False**

(1) isalnum(): Returns True if all characters are alphanumeric (a to z, A to Z ,0 to9)

❖ **Example**

```
txt = "Company12"  
x = txt.isalnum()  
print(x)
```

❖ **Output:** True

(2) isalpha(): Returns True if all characters are only alphabet symbols (a to z,A to Z)

❖ **Example**

```
txt = "CompanyX"  
x = txt.isalpha()  
print(x)
```

❖ **Output:** True

(3) isdigit(): Returns True if all characters are digits only (0 to 9)

❖ **Example**

```
txt = "50800"  
x = txt.isdigit()  
print(x)
```

❖ **Output:** True

(4) islower(): Returns True if all characters are lower case alphabet symbols

❖ **Example**

```
txt = "hello world!"  
x = txt.islower() print(x)
```

❖ **Output:** True

(5) isupper(): Returns True if all characters are upper case alphabet symbols

❖ **Example**

```
txt = "THIS IS NOW!"  
x = txt.isupper()  
print(x)
```

❖ **Output:** True

(6) istitle(): Returns True if string is in title case

❖ **Example**

Check if each word starts with an upper-case letter:

```
a = "HELLO, AND WELCOME TO MY WORLD"  
b = "Hello"  
c = "22 Names"  
d = "This Is %'!?"  
print(a.istitle())  
print(b.istitle())  
print(c.istitle())  
print(d.istitle())
```

❖ **Output:**

```
False  
True  
True  
True
```

(7) isidentifier(): Returns True if the string is an identifier

❖ **Example**

```
a = "MyFolder"  
b = "Demo002"  
c = "2bring"  
d = "my demo"  
print(a.isidentifier())  
print(b.isidentifier())  
print(c.isidentifier())  
print(d.isidentifier())
```

❖ **Output:**

```
True  
True  
False  
False
```

(8) isnumeric(): Returns True if all characters in the string are numeric

❖ **Example**

```
txt = "565543"  
x = txt.isnumeric()  
print(x)
```

❖ **Output:** True

(9) isspace(): Returns True if all characters in the string are whitespaces

❖ **Example**

```
txt = " "  
x = txt.isspace()  
print(x)
```

❖ **Output:** True

❖ **find():**

- ❖ **s.find(substring):** Returns index of first occurrence of the given substring. If it is not available then we will get -1.

- ❖ **Example:**

```
s="Learning C is very easy"
print(s.find("C")) # 9
print(s.find("Python")) # -1
print(s.find("r")) # 3
```

- ❖ **s.find(substring,begin,end):** It will always search from begin index to end-1 index.

- ❖ **Example:**

```
y="armanravipavanshiva"
print(y.find('a',7,15)) #10
```

❖ **count():**

- ❖ **s.count(substring):** It will search through out the string.

- ❖ **s.count(substring,begin,end):** It will search from begin index to end-1 index.

- ❖ **Example:**

```
s="abcabcabcabcadda"
print(s.count('a')) #6
print(s.count('a',3,7)) #2
```

❖ **replace():**

- ❖ **s.replace(old string, new string):** To replace old string with new string.

- ❖ **Example:**

```
s = "Learning C is very difficult"
s1 = s.replace("difficult","easy")
print(s1)
```

- ❖ **Output:**

Learning C is very easy

❖ **split():**

❖ **split(separator):** We can split the given string according to specified separator by using **split()** method.

❖ The default separator is space.

❖ The return type of **split()** method is List.

❖ **Example 1:**

```
s="Arman software"  
l=s.split()  
for x in l:  
    print(x)
```

❖ **Output:**

```
Arman  
Software
```

❖ **Example 2:**

```
s="21-04"  
l=s.split('-')  
for x in l:  
    print(x)
```

❖ **Output:**

```
21  
04
```

❖ **translate() with maketrans function:**

Remove Punctuation using translate() function

❖ **Example:**

```
import string  
test_str = 'Gfg, is best: for! Geeks ;'  
test_str = test_str.translate(str.maketrans(' ', ' ', string.punctuation))  
print(test_str)
```

❖ **Output:**

```
Gfg is best for Geeks
```

❖ **Example:**

```
t="Hi! I am # %Tejas& Thakkar{}"  
x=t.maketrans("", "", "!#%&{}")  
t=t.translate(x)  
print(t)
```

❖ **Output:**

Hi I am Tejas Thakkar

Use a mapping table to replace "S" with "P":

❖ **Example:**

```
txt = "Hello Sam!"  
mytable = txt.maketrans("S", "P")  
print(txt.translate(mytable))
```

❖ **Output:**

Hello Pam!

Use a mapping table to replace many characters:

❖ **Example:**

```
txt = "Hi Sam!"  
x = "mSa"  
y = "eJo"  
mytable = txt.maketrans(x, y)  
print(txt.translate(mytable))
```

❖ **Output:**

Hi Joe!

TUPLE

❖ Definition of Tuple:

- Tuple is exactly same as List except that it is immutable. i.e once we create Tuple object, we cannot perform any changes in that object.
- Hence Tuple is Read only version of List.
- If our data is fixed and never changes then we should go for Tuple.
- Insertion Order is preserved.
- Duplicates are allowed.
- Heterogeneous objects are allowed.
- We can preserve insertion order and we can differentiate duplicate objects by using index.
- Hence index will play very important role in Tuple also.
- Tuple support both +ve and -ve index. +ve index means forward direction (from left to right) and -ve index means backward direction (from right to left)
- We can represent Tuple elements within Parenthesis and with comma separator.
- Parenthesis are optional but recommended to use.

❖ Creation of Tuple:

❖ Creation of empty tuple:

❖ Example:

```
t= ()  
print(type(t))
```

❖ Output:

```
<class 'tuple'>
```

❖ We have to take special care about single valued tuple.

❖ Compulsory the value should ends with comma, otherwise it is not treated as tuple.

❖ Creation of single value tuple:

❖ Example:

```
t=(10)  
print(t)  
print(type(t))
```

❖ **Output:**

```
10
<class 'int'>
```

❖ **Example:**

```
t=(10,)
print(t)
print(type(t))
```

❖ **Output:**

```
(10,)
<class 'tuple'>
```

❖ **Creation of multi value tuple:**

❖ **Example:**

```
t=10,20,30,40
print(t)
print(type(t))
```

❖ **Output:**

```
(10, 20, 30, 40)
<class 'tuple'>
```

❖ **Creation of tuple using tuple() Function:**

❖ **Example:**

```
t=tuple(range(10,20,2))
print(t)
```

❖ **Output:**

```
(10,12,14,16,18)
```

❖ **Accessing Elements of Tuple:**

❖ We can access elements of Tuple either by index or by slice operator

(1) By using Index:

❖ **Example:**

```
t = (10, 20, 30, 40, 50, 60)
print(t[0]) # 10
print(t[-1]) # 60
print(t[100]) # IndexError: tuple index out of range
```

(2) By using Slice Operator:

❖ **Example:**

```
t=(10,20,30,40,50,60)
print(t[2:5])
print(t[2:100])
print(t[::2])
```

❖ **Output:**

```
(30, 40, 50)
(30, 40, 50, 60)
(10, 30, 50)
```

Tuple Vs Immutability:

❖ Once we create tuple, we cannot change its content. Hence tuple objects are immutable.

❖ **Example:**

```
t = (10, 20, 30, 40)
t[1] = 70
```

❖ **Output:**

TypeError: 'tuple' object does not support item assignment

Mathematical Operators for Tuple:

❖ We can apply + and * operators for tuple.

(1) Concatenation Operator (+):

❖ **Example:**

```
t1=(10,20,30)
t2=(40,50,60)
t3=t1+t2
print(t3)
```

❖ **Output:**

```
(10,20,30,40,50,60)
```

(2) Multiplication Operator OR Repetition Operator (*)

❖ **Example:**

```
t1=(10,20,30)
t2=t1*3
print(t2)
```

❖ **Output:**

```
(10,20,30,10,20,30,10,20,30)
```

Important Functions of Tuple:

(1) len(): To return number of elements present in the tuple.

❖ **Example:**

```
t = (10,20,30,40)
print(len(t))
```

❖ **Output:**

4

(2) count(): To return number of occurrences of given element in the tuple

❖ **Example:**

```
t = (10, 20, 10, 10, 20)
print(t.count(10))
```

❖ **Output:**

3

(3) index(): Returns index of first occurrence of the given element. If the specified element is not available then we will get ValueError.

❖ **Example:**

```
t = (10, 20, 10, 10, 20)
print(t.index(10))
print(t.index(30))
```

❖ **Output:**

0

ValueError: tuple.index(x): x not in tuple

(4) sorted(): To sort elements based on default natural sorting order

❖ **Example:**

```
t=(40,10,30,20)
t1=sorted(t)
print(t1)
print(t)
```

❖ **Output:**

```
[10, 20, 30, 40]
(40, 10, 30, 20)
```

❖ We can sort according to reverse of default natural sorting order.

❖ Example:

```
t=(40,10,30,20)
t1 = sorted(t, reverse = True)
print(t1)
```

❖ Output:

```
[40, 30, 20, 10]
```

(5) min() And max() Functions: These functions return min and max values according to default natural sorting order.

❖ Example:

```
t = (40,10,30,20)
print(min(t))
print(max(t))
```

❖ Output:

```
10
40
```

Tuple Packing:

❖ We can create a tuple by packing a group of variables.

❖ Example:

```
a = 10
b = 20
c = 30
d = 40
t = a, b, c, d
print(t)
```

❖ Output:

```
(10, 20, 30, 40)
```

❖ Here a, b, c, d are packed into a Tuple t.

❖ This is nothing but Tuple packing.

Tuple UnPacking:

❖ Tuple unpacking is the reverse process of Tuple packing.

❖ We can unpack a Tuple and assign its values to different variables.

❖ **Example:**

```
t=(10,20,30,40)
a,b,c,d=t
print("a=",a,"b=",b,"c=",c,"d=",d)
```

❖ **Output:**

```
a= 10 b= 20 c= 30 d= 40
```

- ❖ At the time of tuple unpacking the number of variables and number of values should be same, otherwise we will get ValueError.

❖ **Example:**

```
t = (10,20,30,40)
a, b, c = t
```

❖ **Output:**

```
ValueError: too many values to unpack (expected 3)
```

Loop Tuples:

❖ **Loop Through a Tuple**

- ❖ You can loop through the tuple items by using a for loop.

❖ **Example**

- ❖ Iterate through the items and print the values:

```
thistuple = ("apple", "banana", "cherry")
for x in thistuple:
    print(x)
```

❖ **Output:**

```
apple
banana
cherry
```

❖ **Loop Through the Index Numbers:**

- ❖ You can also loop through the tuple items by referring to their index number. Use the range() and len() functions to create a suitable iterable.

❖ Example

- ❖ Print all items by referring to their index number:

```
thistuple = ("apple", "banana", "cherry")
for i in range(len(thistuple)):
    print(thistuple[i])
```

❖ Output:

```
Apple
Banana
Cherry
```

❖ Using a While Loop:

- ❖ You can loop through the list items by using a while loop.
- ❖ Use the len() function to determine the length of the tuple, then start at 0 and loop your way through the tuple items by referring to their indexes. Remember to increase the index by 1 after each iteration.

❖ Example

- ❖ Print all items, using a while loop to go through all the index numbers:

```
thistuple = ("apple", "banana", "cherry")
i = 0
while i < len(thistuple):
    print(thistuple[i])
    i = i + 1
```

❖ Output:

```
Apple
Banana
Cherry
```

❖ **reversed():**

The reversed() method computes the reverse of a given sequence object and returns it in the form of a list.

❖ Example 1:

```
seq_string = 'Python'
# reverse of a string
print(list(reversed(seq_string)))
```

❖ **Output:**

```
['n', 'o', 'h', 't', 'y', 'P']
```

❖ **Example 2:**

```
seq_tuple = ('P', 'y', 't', 'h', 'o', 'n')
# reverse of a tuple object
print(list(reversed(seq_tuple)))
seq_range = range(5, 9)
# reverse of a range
print(list(reversed(seq_range)))
seq_list = [1, 2, 4, 3, 5]
# reverse of a list
print(list(reversed(seq_list)))
```

❖ **Output:**

```
['n', 'o', 'h', 't', 'y', 'P']
[8, 7, 6, 5]
[5, 3, 4, 2, 1]
```

❖ **enumerate():**

In Python, the string is an array, and hence you can loop over it. If you pass a string to `enumerate()`, the output will show you the index and value for each character of the string.

Syntax:

```
enumerate(iterable, start=0)
```

Parameters:

Iterable: any object that supports iteration

Start: the index value from which the counter is to be started, by default it is 0

❖ **Example1:**

Python program to illustrate enumerate function

```
l1 = ["eat", "sleep", "repeat"]
s1 = "geek"
```

creating enumerate objects

```
obj1 = enumerate(l1)
obj2 = enumerate(s1)
```

```
print (list(enumerate(l1)))    # start index will be 0 by default
print (list(enumerate(s1, 2))) # changing start index to 2 from 0
```

❖ **Output:**

```
[(0, 'eat'), (1, 'sleep'), (2, 'repeat')]
[(2, 'g'), (3, 'e'), (4, 'e'), (5, 'k')]
```

❖ **Example 2:**

```
my_str = "abcde"
for i in enumerate(my_str):
    print(i)
```

❖ **Output:**

```
(0, 'a')
(1, 'b')
(2, 'c')
(3, 'd')
(4, 'e')
```

❖ **Using Enumerate object in loops:**

❖ **Example:**

Python program to illustrate enumerate function in loops

```
l1 = ["eat", "sleep", "repeat"]
```

printing the tuples in object directly

```
for ele in enumerate(l1):
    print (ele)
```

changing index and printing separately

```
for count, ele in enumerate(l1, 100):
    print (count, ele)
```

getting desired output from tuple

```
for count, ele in enumerate(l1):
    print(count)
    print(ele)
```

❖ **Output:**

```
(0, 'eat')
(1, 'sleep')
(2, 'repeat')
100 eat
101 sleep
102 repeat
```

0
eat
1
sleep
2
repeat

❖ **Example:**

```
a="Hello how are you"  
for a,b in enumerate(a):  
    print(a,'->',b)
```

❖ **Output:**

0 -> H
1 -> e
2 -> l
3 -> l
4 -> o
5 ->
6 -> h
7 -> o
8 -> w
9 ->
10 -> a
11 -> r
12 -> e
13 ->
14 -> y
15 -> o
16 -> u